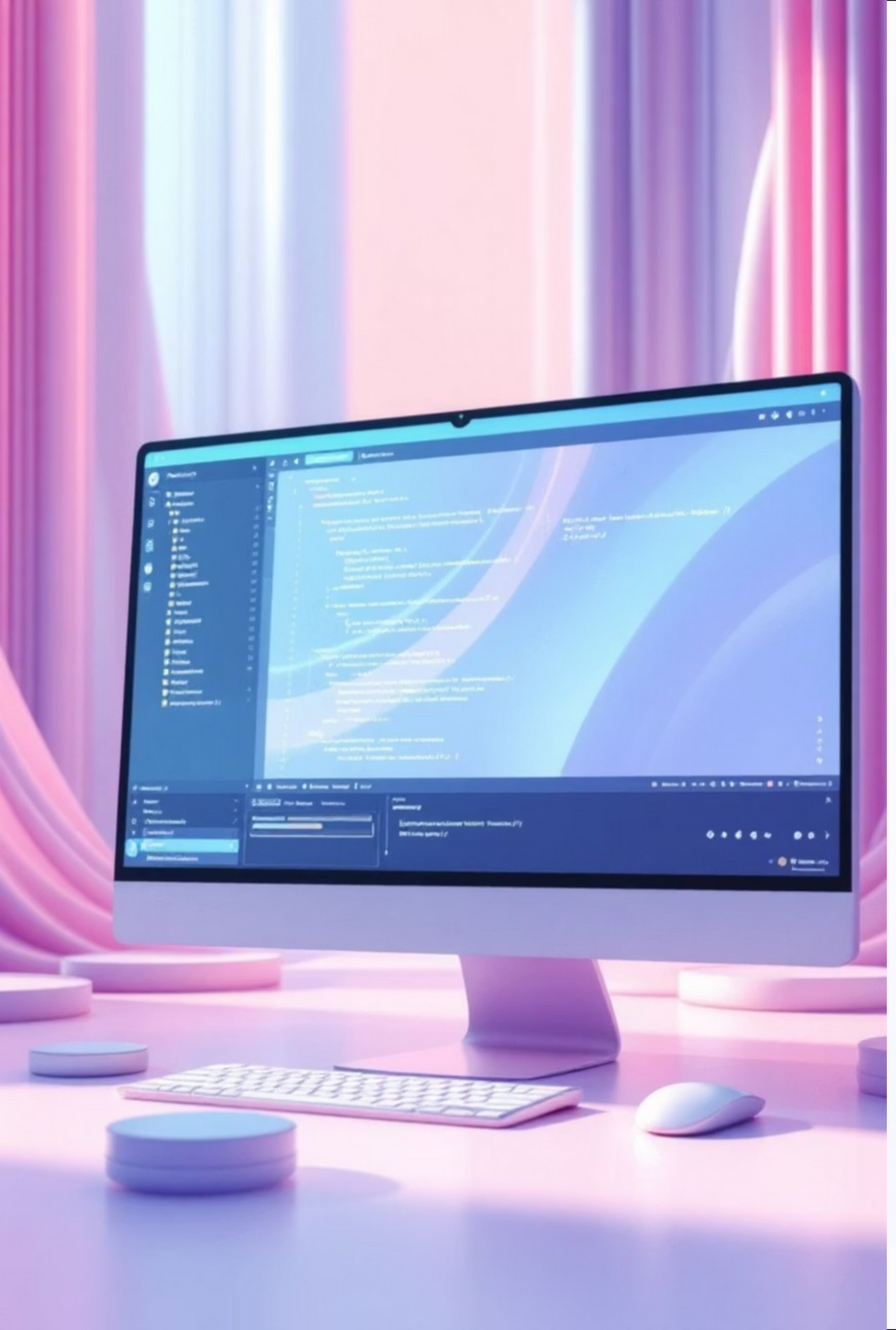


Mountain kin: The King's Demand

This presentation explores the exciting journey of game development using Unity and C#, focusing on practical application and core principles for aspiring developers.

Chapter 1

Introduction to Game Development

Game Concept & Problem Statement

Mountain Kin: The King’s Demand is a single-player action game inspired by Sekiro, designed to address the lack of challenge in modern games by focusing on skill-based combat, precise timing, and strategic decision-making.

Project Objectives

The project aims to develop a playable action game using Unity and C#, featuring responsive combat mechanics, enemy encounters, collision detection, a scoring system, and a clear, user-friendly interface.

Scope & Limitations

The project focuses on a Windows 10/11 playable prototype with keyboard and mouse controls, excluding features such as multiplayer, advanced AI, open-world environments, and high-end graphics due to limited resources.

Project Significance

This project provides hands-on experience in game development by applying Unity, C#, and core gameplay principles, demonstrating effective system integration within a compact action game prototype.

Chapter 2

Technologies & Requirements

Unity Game Engine

The core platform for 2D/3D game creation, enabling visual design and interactive experiences.

C# Programming Language

Essential for scripting game mechanics, UI logic, and various functionalities.

Visual Studio IDE

The integrated development environment for C# coding and seamless Unity integration.

Unity Asset Store

Provides ready-made 2D/3D assets, models, textures, and animations to build the game world efficiently.

Audacity & Audio Tools

Used for creating and editing sound effects to enhance the immersive audio experience of the game.

Defining Functional & Non-Functional Needs

Functional Requirements

- Character movement and interactions.
- Combat actions including attacking, defending, evading, and taking damage.
- Comprehensive score system.
- Basic UI elements (health, stamina, damage feedback, pause menu).
- Reliable game progress saving.

These elements form the core gameplay loop and user interaction.

Non-Functional Requirements

- Smooth real-time performance with stable frame rates.
- Reliable and crash-free gameplay during combat and state transitions.
- Responsive controls with immediate visual and audio feedback.
- Clear and intuitive user interface for player awareness.
- Efficient resource usage to ensure stable gameplay on Windows 10/11.

Ensuring a stable and optimized experience for all players is paramount.

Chapter 3

System Design: Blueprint for Success

This chapter outlines the high-level design of our game, focusing on architectural choices and critical implementation decisions.

01

Character Control & Movement

The system enables responsive character movement and combat actions, ensuring smooth control and precise timing during gameplay.

02

Collision Detection & Interaction

Collision detection handles interactions between the player, enemies, and the environment to ensure accurate combat and movement responses.

03

Score System Design

The score system evaluates player performance based on combat success and objective completion, supporting progression and feedback.

04

User Interface (UI) Design

The UI presents essential gameplay information clearly, providing real-time feedback without interrupting gameplay flow.

05

Game Progress Storage

Game progress is saved and loaded to preserve player state and score, allowing gameplay to continue across sessions.

Chapter 4

Implementation & Results

Development Environment

- **Unity:** The primary game development engine.
- **Visual Studio:** For C# code development and debugging.
- **Unity Asset Store:** For 3D modeling and animation.
- **Audacity:** For sound design and audio editing.

These tools are integrated to provide a robust development workflow.

This section details the practical development process and key outcomes of the project.

Key Code Snippets Explained

```
pdate is called once per frame
y Message | 0 references
Update()

// 1. Luôn tính toán Trọng lực & Nhảy (để không bị treo lơ lửng)
handleGravity();
handleJump();

// 2. Chỉ tính toán di chuyển nếu KHÔNG BỊ KHÓA
if (!lockMovement)
{
    // --- COPY ĐOẠN CODE CŨ CỦA BẠN VÀO ĐÂY ---
    handleCameraRelativeMovement();
    handleRotation();
    handleAnimation();

    appliedMovement.x = isRunPressed ? currentRunMoverment.x : currentMoverment.
    appliedMovement.z = isRunPressed ? currentRunMoverment.z : currentMoverment.
}
else
{
    // --- NẾU BỊ KHÓA THÌ ĐỨNG YÊN ---
    appliedMovement.x = 0;
    appliedMovement.z = 0;

    // Tắt animation chạy bộ nếu đang chạy dở
    if(animator) {
        animator.SetBool("isWalking", false);
        animator.SetBool("isRunning", false);
    }
}

// 3. Di chuyển CharacterController (QUAN TRỌNG: Để trọng lực hoạt động)
if (characterController.enabled)
{
    characterController.Move(appliedMovement * Time.deltaTime);
}
```

```
m BossState { Chasing, Attacking, Cooldown, PreparingLightning, PhaseTransiti
ssState currentState;

rializable]
ss AttackConfig

string name;
int attackIndex;
(0, 100)] public float weight;
int minPhase = 1;


-- REFERENCES ---")]
nsform player;
terWeapon weaponScript;
ssMovement movement;
ssCombat combat;
imator anim;
ssLightningAbility lightning;
kiroStats sekiroStats;

-- COMBO WEIGHT SYSTEM ---")]
t<AttackConfig> attackConfigs = new List<AttackConfig>();

-- SETTINGS ---")]
at attackRange = 2.2f;
at combatCooldown = 3.0f;
loat lastAttackFinishTime;

-- LIGHTNING SKILL CONFIG ---")]
at lightningPrepDistance = 6.0f;
at maxPrepTime = 1.0f;
loat currentPrepTimer;
```

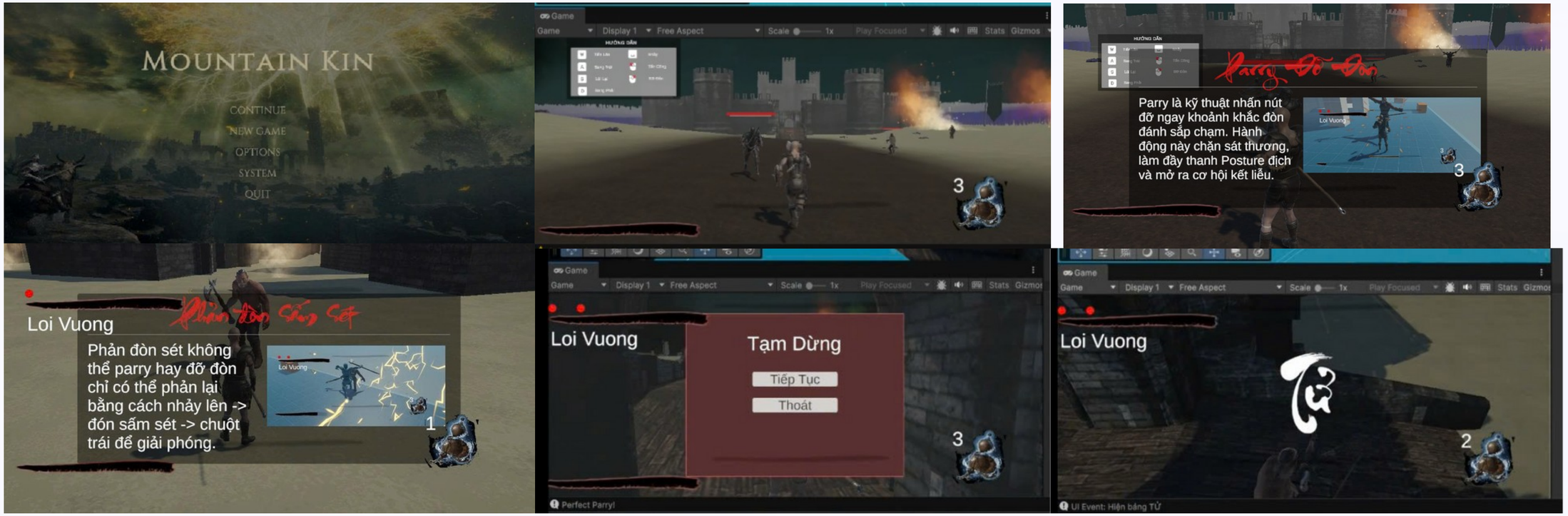
```
if (deathCoroutine != null)
{
    StopCoroutine(deathCoroutine);
    deathCoroutine = null;
}

canReviveInput = false;
isDead = false;
isDeadProcessed = false;

isInvulnerable = true;

// --- SỬA LỖI: BẬT LẠI THANH MÁU ---
if (hpBarMain != null && hpBarMain.transform.parent != null)
{
    hpBarMain.transform.parent.gameObject.SetActive(true);
}
// ---------------------------------

currentHP = maxHP * reviveHPPercent;
currentPosture = 0;
UpdateUI(); // Cập nhật lại thanh máu để nó hiện đúng % máu hồi

if (UIManager.Instance != null) UIManager.Instance.HideDeathPanel();

Animator anim = GetComponent<Animator>();
if (anim != null)
{
    anim.speed = 1f;
    anim.SetBool("isDead", false);
    anim.SetTrigger("Revive");
}
```

```
blic void ApplyDamage(float hpDamage, float postureDamage, bool isBlocked, bool isParried)

  if (isDead || isInvulnerable) return;

  if (isStaggered)
  {
      // Nếu là Boss: Bất tử khi choáng (để bảo vệ logic chuyển phase)
      if (isBoss) return;

      // Nếu là Player: Vẫn đi tiếp xuống dưới để trừ máu (để kích hoạt cơ chế tỉnh lại)
  }

  float finalHPDamage = hpDamage;
  float finalPostureDamage = postureDamage;

  if (isParried) { finalHPDamage = 0; finalPostureDamage *= 0.1f; }
  else if (isBlocked) { finalHPDamage = 0f; finalPostureDamage *= 0.6f; }

  if (finalHPDamage > 0)
  {
      PlayRandomDamageSound();
  }

  currentHP -= finalHPDamage;
  if (currentHP <= 0)
  {
      Die();
      return;
  }
  if (!isStaggered)
  {
      currentPosture += finalPostureDamage;
      if (currentPosture >= maxPosture) BreakPosture();
  }

  UpdateUI();
```

```
d ScanForHit()

RaycastHit hit;
// Kiểm tra va chạm bằng đường thẳng giữa 2 điểm Tip và Base
if (Physics.Linecast(basePoint.position, tipPoint.position, out hit, hitLayer))
{
    GameObject victim = hit.collider.gameObject;

    if (victim != owner && !alreadyHit.Contains(victim))
    {
        HandleHit(victim);
        alreadyHit.Add(victim);
    }
}


d HandleHit(GameObject victim)

// Kiểm tra nếu chủ nhân là Player đánh trúng Enemy
if (owner.CompareTag("Player") && (victim.CompareTag("Enemy") || victim.CompareTag("Boss")))
{
    // 1. Ưu tiên tìm script AI của Boss trước
    if (victim.TryGetComponent<BossAI>(out var bossAI))
    {
        // --- CẬP NHẬT QUAN TRỌNG TẠI ĐÂY ---
        // Truyền cả hpDamage và postureDamage sang cho BossAI xử lý
        bossAI.OnHitByPlayer(hpDamage, postureDamage);

        Debug.Log($"Player chém Boss! HP: {hpDamage}, Posture: {postureDamage}");
    }
    // 2. Nếu không phải Boss (quái thường), tìm Stats chung
    else if (victim.TryGetComponent<SekiroStats>(out var genericStats))
    {
        // Quái thường nhận damage trực tiếp
        genericStats.ApplyDamage(hpDamage, postureDamage, false, false);
    }
}
```

Visualizing Game Features

A collection of screenshots demonstrating the game's core functionalities and user experience.



Visualizing Game Features

Chapter 5

Conclusion & Future Work

Project Achievements

- Successfully implemented character control and movement.
- Accurate collision detection and environmental interactions.
- Functional score system and intuitive UI.
- Robust game progress saving and loading mechanisms.

Acknowledged Limitations

- Lack of multiplayer support.
- Simplified AI opponents.
- Limited level design and graphical complexity.
- Absence of dynamic power-ups.

Future Enhancements

Expand Levels & Graphics

Introduce new, more complex levels and significantly enhance visual fidelity with advanced assets and effects.

Implement Advanced Features

Add dynamic power-ups, diverse enemy types with sophisticated AI, and new gameplay mechanics to enrich the player experience.

Multiplayer & Leaderboards

Integrate online multiplayer functionality and global leaderboards to foster competition and community engagement.

Platform Expansion

Consider porting the game to other platforms like mobile (iOS/Android) or consoles to reach a broader audience.

UEF

ĐẠI HỌC KINH TẾ TÀI CHÍNH

Thanks to our teachers and friends for watching and listening!